Tech Challenge 2 Command Checklist

For Samuel Yeboah • Repeat the project one step at a time in VS Code PowerShell.

Use with your saved Terraform, Kubernetes, Helm, Jenkins and workflow files. This is a command companion to your guide; commands do not replace the configuration files. Setup gaps from the guide are marked explicitly.

Names below use local Docker image tech-2-app, EKS cluster tc2-eks and ECR repository hello-world. Substitute any other resource names consistently before running. Project root means the folder containing app.py and Dockerfile.

☐ 1 Run Flask

Run in: Project root

```
python -m pip install -r requirements.txt
python app.py
```

Check: Open http://localhost:5000. Press Ctrl+C to stop before using port 5000 for Docker.

☐ 2 Build and test Docker

Run in: Project root

Before running: Start Docker Desktop. Save app.py, requirements.txt and Dockerfile.

```
docker build -t tech-2-app .
docker run --rm -p 5000:5000 tech-2-app
```

Check: Open http://localhost:5000. Press Ctrl+C when finished.

☐ 3 Create ECR

Run in: infra folder

Before running: Save your AWS provider and ECR configuration. Ensure AWS CLI credentials are configured.

```
cd infra
terraform init
terraform validate
terraform plan
terraform apply
```

Check: Review the plan, enter yes, and wait for Apply complete. Confirm the repository name in AWS.

Upload the image and create infrastructure

☐ 3 continued Upload to ECR

Run in: PowerShell

Sign Docker into the registry. Run each command below as one command, even if Word wraps the line.

```
aws ecr get-login-password --region us-west-1 | docker login --username AWS --password-stdin 060255765406.dkr.ecr.us-west-1.amazonaws.com
```

Tag your existing local image with the ECR repository address, then push it.

```
docker tag tech-2-app:latest 060255765406.dkr.ecr.us-west-1.amazonaws.com/hello-world:latest
docker push 060255765406.dkr.ecr.us-west-1.amazonaws.com/hello-world:latest
```

Check: The image appears in ECR. The repository must already exist. A tag does not exist error means the exact local image tag being pushed is missing.

☐ 4 Create the network

Run in: infra folder

Before running: Save the VPC, public and private subnets, internet gateway, NAT gateway, route tables and associations.

```
terraform fmt
terraform validate
terraform plan
terraform apply
```

Check: Apply complete. Do not continue past Terraform errors.

☐ 5 Create EKS and worker nodes

Run in: infra folder

Before running: Save cluster IAM, node IAM, EKS and node group configuration.

```
terraform fmt
terraform validate
terraform plan
terraform apply
```

Node settings: t3.small; minimum 1, desired 1, maximum 4. Terraform applies all configuration in the folder, not only the most recently edited file.

Connect to the completed cluster:

```
aws eks update-kubeconfig --region us-west-1 --name tc2-eks
kubectl get nodes
```

Check: The worker node shows Ready.

Deploy and expose the application

☐ 6 Deploy the application with Helm

Run in: k8s folder (the folder containing Chart.yaml and values.yaml)

Save the corrected template in k8s/templates/deployment.yml. See the corrected template at the end of this checklist. Helm fills in {{ ... }} using values.yaml; do not run kubectl apply on this template.

☐ Check the ECR image tag before deploying

In AWS Console, open Amazon ECR in us-west-1 → Private repositories → hello-world → Images. Check the Image tags column. Your uploaded tag is latest; v1 was not found.

In your Helm values file, update the existing image section to match ECR. The repository is the image address; the tag selects the version to download. Save the file before running Helm.

```
image:
  repository: 060255765406.dkr.ecr.us-west-1.amazonaws.com/hello-world
  tag: latest
```

Helm automatically reads values.yaml. If your file is named values.yml instead, explicitly load it:

```
helm lint . -f values.yml
helm upgrade --install hello-world . -f values.yml
```

Run those commands from k8s for values.yml. For the default values.yaml filename, use the commands below.

From k8s/templates, run cd .. first. From the project root, run cd k8s instead. Then run:

```
helm lint .
helm upgrade --install hello-world .
kubectl get pods
kubectl rollout status deployment/hello-world
```

Check: Lint reports 0 charts failed before deploying. Then confirm the pod is Running and ready. A YAML parse error means formatting still needs correction. If Events show ImagePullBackOff with v1: not found, ECR has no image with the requested tag. Correct the tag in the values file, rerun the Helm upgrade command, and check the rollout and pods again. Success means READY 1/1 and STATUS Running.

☐ 7 Test with port forwarding

Run in: Project root

```
kubectl port-forward deployment/hello-world 5000:5000
```

Check: Open http://localhost:5000. Keep the terminal open during testing; press Ctrl+C afterward.

☐ 7 continued Install the AWS Load Balancer Controller

Run in: Git Bash, from any folder. Terraform must already have created the controller IAM role and its permissions.

Purpose: The controller reads your Ingress and creates the AWS Application Load Balancer. Creating its IAM role alone does not install it.

1. Confirm the role and VPC for this cluster:

```
aws iam get-role --role-name tc2-eks-lb-controller --query "Role.Arn" --output text
aws eks describe-cluster --name tc2-eks --region us-west-1 --query "cluster.resourcesVpcConfig.vpcId" --output text
```

Recorded outputs: arn:aws:iam::060255765406:role/tc2-eks-lb-controller and vpc-0f0c5ccbc155e25d5. After rebuilding infrastructure, use the newly returned VPC ID in the installation command. Keep the Terraform files that define module.vpc, module.eks and module.lb_controller_role in infra/. Keep Chart.yaml, values.yaml and templates/ in k8s/, and application.yaml in argocd/. The controller installation below is a terminal command, not an application template; do not paste it into k8s/templates.

2. Add and refresh the Helm repository:

```
helm repo add eks https://aws.github.io/eks-charts
helm repo update
```

3. Copy this entire command into Git Bash. Each backslash continues the same command on the next line:

```
helm upgrade --install aws-load-balancer-controller eks/aws-load-balancer-controller \
  --namespace kube-system \
  --set clusterName=tc2-eks \
  --set region=us-west-1 \
  --set vpcId=vpc-0f0c5ccbc155e25d5 \
  --set serviceAccount.create=true \
  --set serviceAccount.name=aws-load-balancer-controller \
  --set 'serviceAccount.annotations.eks\.amazonaws\.com/role-arn=arn:aws:iam::060255765406:role/tc2-eks-lb-controller'
```

4. Verify the controller, then check the existing Ingress:

```
kubectl get deployment aws-load-balancer-controller -n kube-system
kubectl get ingress hello-world -n default
```

Observed result: The controller showed 2/2 READY and the Ingress received an ALB DNS address. If creating this environment for the first time, deploy the Service and Ingress in the next section before checking the address.

5. Open http:// followed by the ADDRESS returned above (without :5000). Refresh hello-world in Argo CD and check for Synced and Healthy. You reported Healthy after this installation; confirm the greeting in the browser separately.

Troubleshooting lesson: The app pod was 1/1 Running and its rollout succeeded, but Argo CD remained Progressing. The Ingress had a blank Address and no Events. The controller check returned NotFound. Installing the controller resolved the missing ALB address.

☐ 7 continued Apply Service and Ingress

Run in: Project root

For plain YAML files, use the commands below. If Service or Ingress files contain {{ … }} and live in k8s/templates, deploy them with helm upgrade --install hello-world . from k8s instead.

```
kubectl apply -f k8s/service.yaml
kubectl apply -f k8s/ingress.yaml
```

Check: Open your ALB DNS address with http://. Do not add :5000 to the ALB address.

☐ 8 Install Metrics Server and check autoscaling

Run in: Project root

Helm deploys your HPA with the application chart. Metrics Server is separate: it supplies the CPU and memory readings the HPA needs. Run the following commands from any folder.

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
kubectl rollout status deployment/metrics-server -n kube-system
```

Wait about a minute after the rollout completes, then check the readings:

```
kubectl top pods
kubectl get hpa
```

Cluster Autoscaler is a separate installation and is still required for automatic node scaling. Installing Metrics Server does not install Cluster Autoscaler.

Check: CPU and memory numbers appear. If Metrics API not available remains, stop and inspect Metrics Server before continuing. The HPA limit of 1–12 pods applies across the cluster; it does not enforce three pods per node.

Run Jenkins and prepare GitHub Actions

☐ 9 Validate the Helm chart

Run in: Project root

Your chart is in k8s: Chart.yaml, values.yaml and templates/deployment.yml. It was deployed in Step 6; check it again after any changes.

```
helm lint k8s
```

If already inside the chart folder, use helm lint . instead. Check: 0 chart(s) failed.

☐ 10 Run the Jenkins pipeline

Run in: Jenkins dashboard

Before running: Configure Jenkins credentials, tools and the saved pipeline for the main branch. Any existing Deployment must be compatible with Helm ownership before the pipeline installs the release.

Open your pipeline and click Build Now. Check the Console Output for Finished: SUCCESS.

Verify from PowerShell:

```
helm list -n default
kubectl rollout status deployment/hello-world
kubectl get pods
```

☐ 11 Connect the new repository and upload GitOps

Run in: Project root in the Git repository

Run from your tech-2a project folder. The old GitHub repository was deleted and recreated. These commands upload committed files; commit intended changes first and keep credentials and Terraform state out of Git.

```
git remote set-url origin https://github.com/skysamuels-art/tech-2a-app.git
git switch main
git push -u origin main
git switch -c gitops
git push -u origin gitops
```

If gitops already exists, use git switch gitops, then git push -u origin gitops. Refresh GitHub and select gitops from the branch dropdown. If Repository not found appears, confirm the repository exists and Git is signed in with an account that has access.

☐ 11 continued Configure GitHub OIDC in AWS

Run in: AWS Console. OIDC gives GitHub Actions temporary AWS credentials without storing AWS access keys in GitHub. This console setup replaces the earlier Terraform OIDC instructions; do not create duplicate resources with Terraform.

1. Open IAM → Identity providers. Reuse token.actions.githubusercontent.com if it exists. Otherwise choose Add provider → OpenID Connect and enter:

Provider URL: https://token.actions.githubusercontent.com
Audience: sts.amazonaws.com

2. Open IAM → Roles → Create role → Web identity. Select the GitHub provider and sts.amazonaws.com audience. Enter these restrictions:

GitHub organization: skysamuels-art
GitHub repository: tech-2a-app
GitHub branch: gitops
Role name: tech-2a-github-role

Create the role without a permissions policy initially. The organization field accepts your GitHub username. Restrict the trust to this repository and branch.

3. Open the role → Permissions → Add permissions → Create inline policy → JSON. Paste the policy below and save it as tech-2a-ecr-push.

```
{
 "Version": "2012-10-17",
 "Statement": [
  {
   "Effect": "Allow",
   "Action": "ecr:GetAuthorizationToken",
   "Resource": "*"
  },
  {
   "Effect": "Allow",
   "Action": [
    "ecr:BatchCheckLayerAvailability",
    "ecr:GetDownloadUrlForLayer",
    "ecr:BatchGetImage",
    "ecr:InitiateLayerUpload",
    "ecr:UploadLayerPart",
    "ecr:CompleteLayerUpload",
    "ecr:PutImage"
   ],
   "Resource": "arn:aws:ecr:us-west-1:060255765406:repository/hello-world"
  }
 ]
}
```

☐ 11 continued Create and push the AWS connection test

Run in: VS Code on your local computer, using Git Bash. Go to the project root and switch to gitops:

```
cd ~/Desktop/cloud_projects/tech-2a
git switch gitops
```

In VS Code Explorer, create two folders and one file: .github is a folder; workflows is a folder inside .github; check-aws.yml is a file inside workflows. Do not create check-aws.yml as a folder.

File location: tech-2a/.github/workflows/check-aws.yml

Open check-aws.yml, paste the full workflow below, and press Ctrl+S:

```
name: Check AWS connection

on:
  push:
    branches:
      - gitops

permissions:
  id-token: write
  contents: read

jobs:
  check-aws:
    runs-on: ubuntu-latest

    steps:
      - name: Sign in to AWS
        uses: aws-actions/configure-aws-credentials@v6.3.0
        with:
          role-to-assume: arn:aws:iam::060255765406:role/tech-2a-github-role
          aws-region: us-west-1

      - name: Check AWS identity
        run: aws sts get-caller-identity
```

id-token: write lets the workflow request an OIDC token. The role controls what AWS actions it can perform. This workflow tests authentication only; it does not build or push an image.

From the project root, upload the saved file:

```
git add .github/workflows/check-aws.yml
git commit -m "Test GitHub Actions AWS connection"
git push -u origin gitops
```

Check: Open GitHub → tech-2a-app → Actions → Check AWS connection. A green run and account 060255765406 with the assumed role confirm AWS authentication. Image building and pushing follow in Step 12.

☐ 11 continued Troubleshoot GitHub OIDC authentication

Use this section if Sign in to AWS reports Not authorized to perform sts:AssumeRoleWithWebIdentity. The workflow requests a role, and the role trust policy must match the identity GitHub sends.

1. Confirm the workflow uses this role directly. With this setting, AWS_ROLE_ARN in GitHub Secrets is not used:

role-to-assume: arn:aws:iam::060255765406:role/tech-2a-github-role

2. In your existing workflow file, add the following step immediately below steps: and before Sign in to AWS. Preserve spaces and indentation. Do not copy Markdown asterisks, escaped backticks or HTML entities into YAML.

```
- name: Check GitHub identity
  uses: actions/github-script@v7
  with:
    script: |
      const token = await core.getIDToken('sts.amazonaws.com');
      const claims = JSON.parse(
        Buffer.from(token.split('.')[1], 'base64url').toString()
      );
      core.info(`Subject: ${claims.sub}`);
      core.info(`Audience: ${claims.aud}`);
```

This diagnostic prints Subject and Audience only. Do not print or share the complete authentication token. Keep id-token: write and contents: read in the workflow permissions.

3. Save the file. Run from your project root in Git Bash:

```
git switch gitops
git add .github/workflows/
git commit -m "Check OIDC identity and fix workflow formatting"
git push origin gitops
```

4. Open GitHub → Actions → newest run → check-aws → Check GitHub identity. Copy the complete value after Subject:. Check that Audience is sts.amazonaws.com. The sign-in step may still fail until the trust policy is corrected.

5. Open AWS → IAM → Roles → tech-2a-github-role → Trust relationships → Edit trust policy. Under token.actions.githubusercontent.com:sub, replace the two duplicate entries with one quoted entry containing the exact Subject copied from the log. Keep the surrounding brackets.

Finding in this project: GitHub sent numeric IDs after the username and repository name using @. The original policy contained only the names. Copy the current Subject exactly, including IDs and :ref:refs/heads/gitops. Do not guess IDs or broaden the policy to all repositories.

6. Remove the comma after the final quoted Subject entry. A trailing comma before] is invalid JSON and caused Failed to update trust policy. Duplicate identical entries were unnecessary but were not the authentication mismatch.

7. Click Update policy. Then open the failed GitHub run → Re-run jobs → Re-run failed jobs. No new Git commit is needed for a trust-policy-only change.

Success check: Sign in to AWS is green; Check AWS identity shows account 060255765406 and the assumed tech-2a-github-role. Verify this before Step 12. Authentication success does not yet confirm image push permissions or deployment.

☐ 12 Upload the GitHub Actions workflow

Run in: Project root on gitops

Before running: Save .github/workflows/build-image.yaml. It must build and push an image, then commit the new image tag to the Helm values file. Use your actual IAM role ARN.

```
git add .github/workflows/build-image.yaml
git commit -m "Add GitHub Actions image build workflow"
git push origin gitops
```

Check: GitHub Actions succeeds, the image appears in ECR, and the Helm values file has the new image tag.

☐ 13 Install Argo CD

Run in: Git Bash, from any folder. Argo CD must be installed before Kubernetes can accept an Application resource.

```
aws eks update-kubeconfig --region us-west-1 --name tc2-eks
kubectl create namespace argocd
kubectl apply -n argocd --server-side -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
kubectl get pods -n argocd
```

If namespace creation says AlreadyExists, continue. Allow several minutes for startup. Confirm Running status and matching READY counts before opening the dashboard.

Troubleshooting: no matches for kind Application means the Argo CD resource definition is missing from the connected cluster. Confirm the cluster context and complete installation above; then apply application.yaml again.

☐ 13 continued Fix pods stuck in Pending

If port forwarding says pod is not running, inspect the server pod events:

```
kubectl get nodes
kubectl get pods -n argocd
kubectl describe pods -n argocd -l app.kubernetes.io/name=argocd-server
```

Finding in this project: Events reported 0/1 nodes are available: 1 Too many pods. The single worker node had no remaining pod slots. This is a node capacity issue, not a port-forwarding syntax error.

Your Terraform EKS module is version ~> 20.0. Keep the node-group settings below. Its managed node group ignores desired-size changes after creation, so editing desired_size alone did not add the second node.

```
instance_types = ["t3.small"]
min_size       = 1
desired_size   = 2
max_size       = 4
```

To request the second node now: AWS Console in us-west-1 → EKS → Clusters → tc2-eks → Compute → select the node group → Edit scaling configuration. Set minimum 1, desired 2, maximum 4, then save. Allow several minutes for the node to join.

```
kubectl get nodes
kubectl get pods -n argocd
```

Check: Two nodes show Ready; waiting Argo CD pods schedule automatically. Do not retry port forwarding until argocd-server shows 1/1 Running. If a pod remains Pending, inspect its latest Events again.

The Terraform autoscaler_role creates IAM permissions only; it does not install Cluster Autoscaler. Maximum size 4 is a limit, not a request to launch four nodes. Automatic scaling still requires a working Cluster Autoscaler installation.

If argocd-dex-server remains in CrashLoopBackOff, inspect its previous logs. Do not assume adding a node fixes this separate error:

```
kubectl logs -n argocd deployment/argocd-dex-server --previous --tail=50
```

☐ 13 continued Open Argo CD and connect GitHub

Run in: Git Bash, from any folder. Start port forwarding and leave this terminal open:

```
kubectl port-forward service/argocd-server -n argocd 8084:443
```

Open https://localhost:8084 in your browser. In a second Git Bash terminal, retrieve the initial password:

```
kubectl get secret argocd-initial-admin-secret -n argocd -o jsonpath="{.data.password}" | base64 --decode
```

Sign in as admin with the decoded password. Do not share it or commit it. This command requires the initial administrator secret to still exist.

Create the GitHub token before connecting the repository: GitHub profile picture → Settings → Developer settings → Personal access tokens → Fine-grained tokens → Generate new token.

```
Token name: tech-2a-argocd
Resource owner: skysamuels-art
Repository access: Only select repositories → tech-2a-app
Repository permissions: Contents → Read-only
Expiration: Choose an expiration date
```

Generate and copy the token. Use it only in the repository connection password field below. The Argo CD dashboard login uses admin and its initial password; the GitHub token is a separate credential. Never commit or share either secret.

Open Settings → Repositories → Connect Repo. Select HTTPS and type git. Enter the values below, then click Connect:

```
Repository URL: https://github.com/skysamuels-art/tech-2a-app.git
Username: skysamuels-art
Password: your GitHub personal access token with read access to this repository
```

Check: Repository connection status is Successful. This connection lets Argo CD read the private Helm chart. GitHub Actions AWS OIDC authentication does not grant Argo CD access to your private GitHub repository.

☐ 14 Create the Argo CD Application file

In VS Code, create argocd beside k8s in the project root, not inside k8s. Create application.yaml inside argocd. The paths should be:

```
tech-2a/k8s/Chart.yaml
tech-2a/k8s/values.yaml
tech-2a/k8s/templates/
tech-2a/argocd/application.yaml
```

The Application points to the k8s chart on the gitops branch. Ensure the chart is committed and pushed there. Pause Jenkins deployments for the same application while Argo CD manages it.

☐ 14 continued Save and apply the Application

Save the following in argocd/application.yaml:

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: hello-world
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/skysamuels-art/tech-2a-app.git
    targetRevision: gitops
    path: k8s
    helm:
      releaseName: hello-world
  destination:
    server: https://kubernetes.default.svc
    namespace: default
  syncPolicy:
    automated:
      selfHeal: true
```

This uses k8s/values.yaml by default. If your actual file is values.yml, add valueFiles: [values.yml] under helm, aligned with releaseName.

From the project root, run:

```
kubectl apply -f argocd/application.yaml
kubectl get applications -n argocd
```

If already inside the argocd folder, use this path instead:

```
kubectl apply -f application.yaml
```

Expected first apply: application.argoproj.io/hello-world created. A later apply may say configured or unchanged. Then check Applications → hello-world in the dashboard for Synced and Healthy. Creating the Application object alone does not prove deployment succeeded.

Save the setup in Git. From the project root on gitops:

```
git add argocd/application.yaml
git commit -m "Add Argo CD application"
git push origin gitops
```

Purpose: Argo CD reads the Helm chart from Git and applies it to EKS. A successful Check AWS connection workflow only confirms authentication. To complete CI/CD, the build workflow must push a new image to ECR and update its tag in the Git-tracked Helm values file.

Troubleshooting Synced but Degraded: Synced means the application resources match the desired manifests in Git. Degraded means at least one managed resource is unhealthy. The application card alone does not identify the failing resource. Open hello-world and select the red resource to inspect its details.

Run from any Git Bash folder to inspect the application and pods:

```
kubectl get pods -n default
kubectl describe deployment hello-world -n default
kubectl get events -n default --sort-by=.metadata.creationTimestamp
```

Review the latest warnings before making changes. Do not assume this is another image-tag or node-capacity problem without the Events output. Confirm Synced and Healthy before claiming the deployment is working.

☐ 14 continued Fix the Argo CD image tag

Observed error: Argo CD showed Synced but Degraded. Pod Events reported hello-world:v1: not found, followed by ErrImagePull and ImagePullBackOff. One older pod was Running while the replacement pod could not download its image. A running older pod does not prove the new rollout succeeded.

1. In AWS Console, select us-west-1 → ECR → Private repositories → hello-world → Images. Confirm the tag latest exists before selecting it in Git.

2. From the project root in Git Bash, switch to the branch Argo CD watches before editing:

```
git switch gitops
```

If your terminal is in tech-2a/argocd, run cd .. first to return to the project root. If Git refuses the branch switch because of local changes, resolve that message before continuing; do not discard your work.

3. Open k8s/values.yaml in VS Code. Update the existing image section and save. Keep your other settings:

```
image:
  repository: 060255765406.dkr.ecr.us-west-1.amazonaws.com/hello-world
  tag: latest
replicaCount: 1
resources:
  requests: { cpu: 100m, memory: 64Mi }
  limits: { cpu: 250m, memory: 128Mi }
hpa:
  minReplicas: 1
  maxReplicas: 12
  cpuTarget: 50
  memoryTarget: 50
```

The HPA maximum of 12 applies across the cluster; it does not enforce three pods on each node.

4. Commit and push the saved values file from the project root:

```
git add k8s/values.yaml
git commit -m "Fix ECR image tag to latest"
git push origin gitops
```

If your actual file is values.yml, use k8s/values.yml in git add instead. Argo CD must explicitly load that filename through helm.valueFiles, as described in Step 14. If Git says nothing to commit, check the file on GitHub with the gitops branch selected and confirm tag: latest is already there.

5. In Argo CD, open hello-world → Refresh. Allow automatic sync to apply the new Git revision. Then run:

```
kubectl rollout status deployment/hello-world -n default --timeout=120s
kubectl get pods -n default
```

Success check: Rollout reports successfully rolled out, the new application pod is 1/1 Running, and Argo CD shows Synced and Healthy. These are expected results to verify, not confirmation that the latest fix has already succeeded.

If the rollout times out, inspect the latest pod Events again. Do not delete the running pod just to clear the display. Argo CD reads GitHub, so a local edit or manual Helm change alone does not correct the desired image in Git.

This latest-tag correction restores a valid image reference. For future automated releases, CI must update a distinct image tag or digest in Git; pushing different image bytes under the same latest tag alone does not trigger an Argo CD rollout.

☐ 15 Test the complete workflow

Run in: Project root on gitops

Before running: Change the greeting in app.py.

```
git add app.py
git commit -m "Update application greeting"
git push origin gitops
```

Check in order: GitHub Actions succeeds → Helm image tag changes in Git → Argo CD syncs → updated greeting appears at the ALB address.

☐ 16 Save submission evidence

Run in: GitHub and browser

☐ Jenkins successful build screenshot
☐ GitHub Actions successful run screenshot
☐ Argo CD Synced and Healthy screenshot
☐ Updated greeting with the ALB address visible
☐ README and private repository access shared with your mentor

Corrected deployment template

Save as k8s/templates/deployment.yml. Replace the full file, preserve indentation, and use spaces rather than tabs. The resources expression must not contain asterisks.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello-world
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app: hello-world
  template:
    metadata:
      labels:
        app: hello-world
    spec:
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: kubernetes.io/hostname
          whenUnsatisfiable: ScheduleAnyway
          labelSelector:
            matchLabels:
              app: hello-world
      containers:
        - name: app
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          ports:
            - containerPort: 5000
          resources:
            {{- toYaml .Values.resources | nindent 12 }}
```

In k8s/values.yaml, image.repository must be 060255765406.dkr.ecr.us-west-1.amazonaws.com/hello-world and image.tag must match your uploaded tag, currently latest. Keep replicaCount and resources defined.